

Verification Continuum™

Siloti® Replay Simulation User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Audience	4
Customer Support	4
Synopsys Statement on Inclusivity and Diversity	5

1. Introduction	6
Overview	6
Verdi Power Analysis Acceleration Overview	6

2. Generate Target FSDB From Source FSDB With Replay Simulation	7
Overview	7
Direct-Mapping Flow	8
Usage	9
Required Data	10
Normal Flow	11
Phase 1: Extraction	11
Phase 2: Compilation	11
Phase 3: Simulation	11
Sanity Check	12
Remap Flow	13
Configuration File	14
Text Mapping File	17
Form Bus Flow	17
Simulation Compile Script Examples	18
Simulation Run Script Examples	19
Generate Template Files	20
Supported Platforms and Simulator	20
Practice	20
Step1: Generate Mapping File	21
Step2: Replay Simulation	22
Known Problems and Limitations	23

Preface

The Siloti Replay Simulation User Guide provides detailed description of every command and option of Siloti Replay Simulation.

Audience

The audience for this manual includes engineers who are familiar with languages and tools used in design and verification and are frequently required to complete Engineering Change Orders (ECOs) on gate-level netlists.

This document assumes that you have a basic knowledge of the platform on which your version of the Verdi platform runs: UNIX or Linux and that you are knowledgeable in Verilog or VHDL, simulation software, and digital logic design.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetsynopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction

In this chapter:

- [Overview](#)
- [Verdi Power Analysis Acceleration Overview](#)

Overview

Siloti Replay Simulation is a technology that allows you to use simulation results from other abstractions as the input data to the re-run full/partial simulation. Siloti Replay Simulation allows you to select the time range or scopes as the target and use the input from other simulation runs to feed into the simulator and generates simulation results of the current target scope or time range.

Verdi Power Analysis Acceleration Overview

Verdi Power Analysis Acceleration uses Siloti Correlation and Siloti Replay Simulation technologies to help user run the gate level simulation and dump necessary signals waveform for power analysis tools to calculate the power consumption.

To obtain the most accurate power analysis results, you must generally use the later design stage of simulation result as the switching activity data for the power analysis. However, due to the runtime of gate level simulation that takes long time, users face the challenges to obtain the gate level simulation for the power analysis. With Siloti Correlation technology, users can use the RTL simulation result to correlate to the gate level design. Siloti Replay Simulation takes the input from the RTL simulation and run the gate level simulation to obtain the gate level simulation result for the power analysis.

You can also refer to the PowerReplay User Guide. The PowerReplay product allows early and accurate power estimation and also leverages multiple technologies including Siloti Replay.

2

Generate Target FSDB From Source FSDB With Replay Simulation

In this chapter:

- [Overview](#)
 - [Direct-Mapping Flow](#)
 - [Supported Platforms and Simulator](#)
 - [Practice](#)
 - [Known Problems and Limitations](#)
-

Overview

The Verdi Automated Debug Platform provides a methodology that automatically generates the target waveform (with specific scope, either RTL or gate-level design) from the original environment. The most frequent application is generating a gate-level waveform for a chip design from the RTL design environment without the need to bring up the gate-level environment. The waveform generation methodology also reduces the effort to perform gate-level power analysis from weeks to hours using established EDA technology and power analysis flow. This major reduction in effort and increase in productivity enables you to analyze power characteristics much earlier in the design flow than is practically possible using the traditional gate-level analysis. Moreover, the methodology produces waveforms identical (or nearly identical) to those generated by the gate-level simulation. Consequently, the design can be analyzed and optimized iteratively throughout the post-synthesis design flow, enabling much earlier detection and easier resolution of power issues.

The Replay methodology provides the following features:

- Generates accurate gate-level waveforms automatically.
- Enables you to correlate and analyze results in the RTL environment without bringing up the entire gate-level environment.

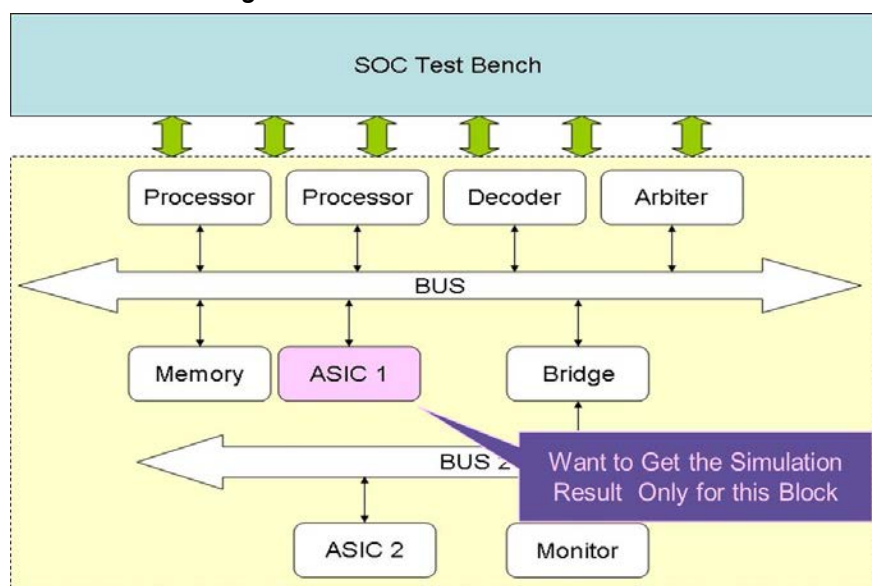
- Eliminates the necessity to simulate from time zero for every power analysis run.
- Imposes no changes on the established power analysis methodology and tool flow.

The methodology employs the main tool feature “replay simulation” to rerun gate-level simulation with RTL simulation waveforms as the input stimuli or to rerun RTL simulation with gate-level simulation waveforms as the input stimuli. Replay simulation provides a direct mapping flow to map the signals from the RTL to the gate-level and vice versa. For power analysis, this technology is used to map the RTL signal waveform to the gate-level signals to drive the simulation again.

Direct-Mapping Flow

In a complex SOC design, replay simulation enables you to obtain gate-level or RTL simulation results of a specific block, say `ASIC 1` block in the figure below. The following sections explain how to use replay simulation through the `wisim` utility to focus on direct mapping to the `ASIC 1` block.

Figure 1 SoC Design

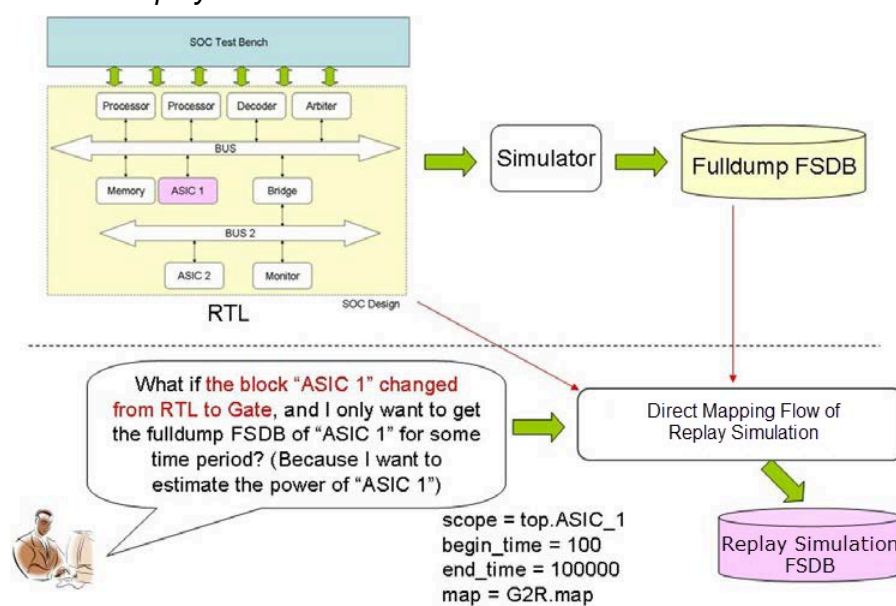


The `wisim` utility provides a simple way to simulate a target design, either RTL or gate-level design, using a source FSDB file. The `wisim` utility maps the signals from the RTL to the gate-level design or from the gate-level design to the RTL where the mapping file contains both flip-flop mapping and input mapping. Then, the `wisim` utility automatically generates stimuli that can be given as input to the target design simulation. This means

you can use the RTL testbench to directly generate a gate-level FSDB file without modifying the testbench by the correlation flow.

The following figure shows the complete replay simulation. Consider that you already have the gate-level design and an FSDB file for the RTL design. You may generate a new FSDB file that shows the gate-level waveform and/or a SAIF file to estimate the power consumption for the ASIC 1 block. The gate-level design and RTL-FSDB files can be used with the `wisim` utility to generate the new testbench for the specified scope. Then, the simulator is invoked to run this partial simulation to generate the on-demand FSDB file and/or SAIF file.

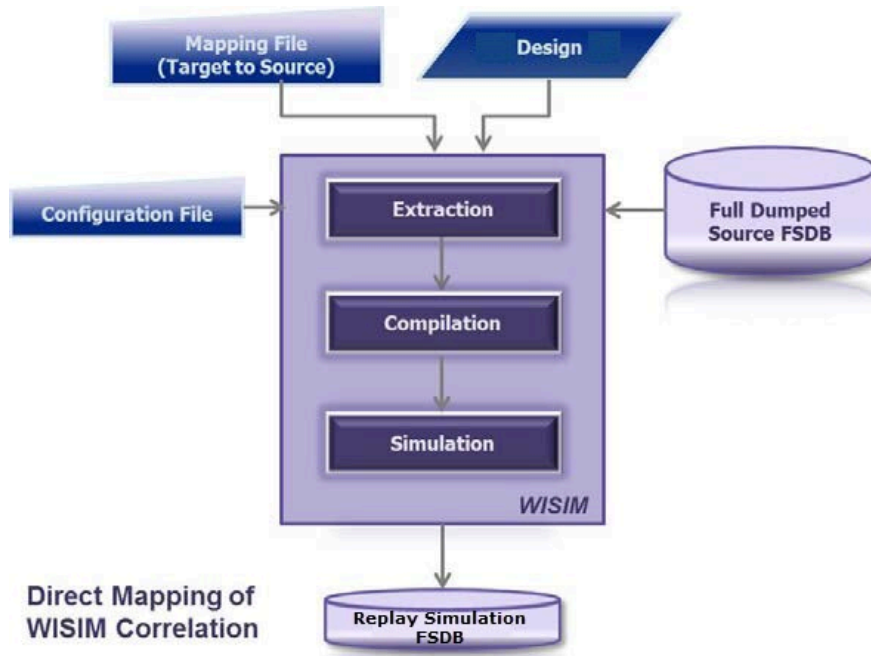
Figure 2 *Replay Simulation Correlation Flow*



Usage

To start running the `wisim` utility, you must prepare the input and output files.

Figure 3 Input and Output Files of Direct Mapping for Replay Simulation Correlation



Required Data

As shown in the figure, the required data for the direct mapping flow includes the following:

- **Design Files:** Design files that are imported to the Replay Simulation.

Note:

It is recommended to replace the symbol library with a simulation model.

- **Source FSDB File:** Contains signal values required by the source signals in the mapping file.
- **Mapping File:** Contains the signal list of target signals mapped to the source signals (See the Text Mapping File section for text mapping file details).
- **Configuration File:** Contains the specifications of the target scope, time range, units, and scripts (See the Configuration File section for details).
- **SDF file:** The Standard Delay Format (SDF) file stores the timing data that includes the delay information. This file is optional for the direct mapping flow.

Note:

The `wisim` utility forces all target signals in the mapping file if their mapped source signals are found in the source FSDB file. The `wisim` utility

generates value changes in order according to the input source FSDB file that contains the event sequence information when two mapping signals have value changes at the same time.

Normal Flow

The direct mapping flow of replay simulation includes three phases: extraction, compilation, and simulation. Each phase has a corresponding option to be executed with the *wisim* utility.

Phase 1: Extraction

This phase extracts the design and generates the new testbench using the `-replay_extract` option on the *wisim* command line. Also, to import a file to set the configurations, use the `-config` option as follows:

```
%> wisim -top "work.system" -config config_file -replay_extract
```

For more details on the `-config` option, see the Configuration File section.

By default, all files generated by the *wisim* utility are collected in `whatIf/` directory. You can modify the directory name by using the `-target <target_dir_name>` option for the *wisim* utility.

Optionally, you can use the `-saif` option in the extraction phase to enable the SAIF file dumping in the simulation phase. The `wi.saif` file in the replay simulation directory is generated after completing the flow. Currently, only the VCS simulator is supported to generate the SAIF file. For example:

```
%> wisim -top "work.system" -config config_file -replay_extract -saif
```

In addition, you can set the `WISIM_DUMP_MAP` environment variable to `1` before the extraction phase to dump the `wi_dump_map.txt` file of the parsed mapping table. You can check the mapping results after executing the *wisim* utility.

Phase 2: Compilation

This phase compiles the extracted design and testbench using the `-replay_compile` and `-config` options specified on the *wisim* command line. The syntax is:

```
%> wisim -config config_file -replay_compile
```

Phase 3: Simulation

This phase runs the simulation using the `-replay_sim` and `-config` options specified on the *wisim* command line. The syntax is:

```
%> wisim -config config_file -replay_sim
```

To convert all mapping signals containing unknown values to '0' or '1' set the constant value `-x_to_const` option with value '0' or '1' in the simulation phase and then pass the value to the target design. For example, to convert the unknown value to 0 using the following command:

```
%> wisim -config config_file -replay_sim -x_to_const 0
```

The `wisim` utility also provides the following dumping option settings in the simulation phase for users to customize the simulation without re-compiling:

- **+WI_NO_DUMP:** Disables FSDB file dumping during the whole simulation.
- **+WI_NO_SDF:** Does not perform SDF annotation.
- **+WI_NO_SAIF:** Does not generate the SAIF file.
- **+WI_DUMP_VCD:** Enables the VCD file dumping.

Sanity Check

You can perform sanity check before extraction phase to measure the quality of input files in advance. For example:

```
%> wisim -top "work.system" -config config_file -sanity_check
```

The main function of sanity check is to validate the accuracy of the mapping file. The following is the checklist for sanity check:

1. All left-hand-side signals in the mapping file should be in the imported design.
2. All right-hand-side signals in the mapping file should be in the FSDB.
3. All boundary input/in out signals for the target scope must be in the mapping file. (By default, the checked scope is the target scope in configuration file. If you want to change the checked scope, use `set Boundary_Check_Scope = NewScopeName` in the configuration).
4. The compilation/simulation script conforms to the requirement of Replay Simulation (This is not turned on by default. You need to add the following line to configuration file `set Checking_Rule= +script` to turn it on).

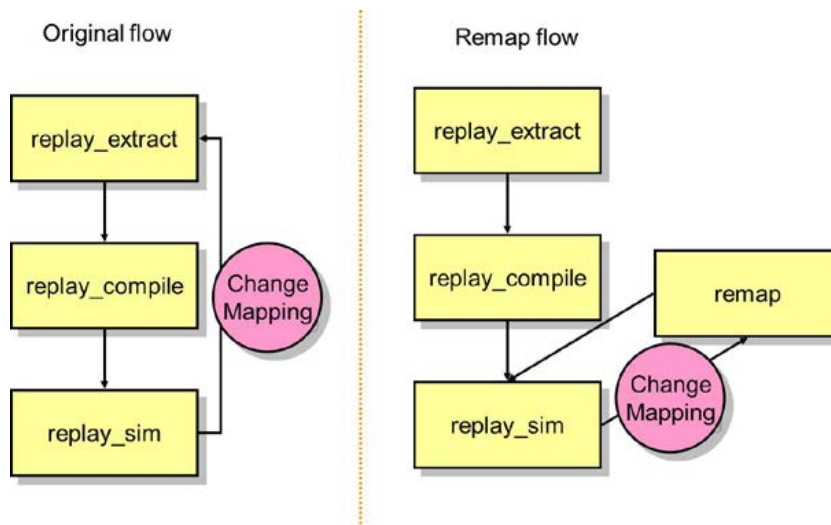
After sanity check, the output log is collected in `whatIf.check/`, which contains the following files:

- `wi_check.log`: Specifies the log file for sanity check.
- `wi_newmap.txt`: If there are any incorrect signal pairs in the mapping file, they are collected in the file.
- `wi_unmap.log`: If there are any dropped signal pairs during sanity check, they are collected in this file.

Remap Flow

The direct mapping flow also provides an efficient method to apply re-mapping requirements.

Figure 4 *Replay Simulation Remap Flow*



When updates exist in the mapping file and no updates exist in the design files or the FSDB file, you may run the original correlation flow again from the extraction phase to the simulation phase. The `wisim` utility provides an alternate and simple way to re-map the mapping file without spending time to extract the extracted design and generate the generated testbench again.

Note:

The normal direct mapping flow needs to be executed at least once to extract the target design and generate the testbench for the first time.

To notify replay simulation analysis for using the remap flow that does not extract the design again and execute the simulation phase, use the `-remap` option in the extraction phase. For example:

```
%> wisim -top "work.system" -config config_file -replay_extract  
-remap  
%> wisim -config config_file -replay_sim
```

Configuration File

The configuration file format for the direct mapping flow is shown in the following example:

```
set FSDB = ./rtl.fsdb
set Scope = system.i_cpu.i_ALUB
set Map = ./map.file
set Begin_Time = 1000
set End_Time = 3000
set Time_Unit = ns
set Simulation_Compile_Script = ./ius_wi_compile.rc
set Simulation_Run_Script = ./ius_wi_run.rc
set delay = ./delay.map
set SDF = "../Design/CPU_max.sdf" "system.i_cpu"
set Design = ./system_load.command
set Exclude_Module_File = ./exc_mod.txt
set FSDB_HIER_MAP = RTL_Hierarchy_in_Mapping_file    RTL_Hierarchy_in_FSDB
```

The required inputs are highlighted in red and the optional inputs are highlighted in green. The following table shows the configurations for the direct mapping flow of replay simulation:

Configuration	Description and Example for the Configuration
FSDB	The dumped FSDB file of the source design. Example: set FSDB = ./rtl.fsdb
Scope	The target scope of the entire design hierarchy. Example: set Scope = system.i_cpu.i_ALUB
Map	List the target signals and map to source signals (see the Text Mapping File section for details). Example: set Map = ./map.file
Begin_Time	The begin time for replay simulation. NOTE: If the begin time is not specified, the minimum time of the input FSDB file is used as the begin time automatically. Example: set Begin_Time = 1000
End_Time	The end time for replay simulation. NOTE: If the end time is not specified, the maximum time of the input FSDB file is used as the end time automatically. Example: set End_Time = 3000

Time_Unit	The time unit for Begin_Time and End_time. NOTE: If either the begin time or the end time is specified, the time unit must be specified in the replay simulation configuration file. Example: <pre>set Time_Unit = ns</pre>
Simulation_Compile_Script	The simulator compilation script (see the Simulation Compile Script Examples section for an example). Example: <pre>set Simulation_Compile_Script = ./vcs_wi_compile.rc</pre>
Simulation_Run_Script	The simulator simulation script (see the Simulation Run Script Examples section for an example). Example: <pre>set Simulation_Run_Script = vcs_wi_sim.rc</pre>
delay	Optional. The configuration specifies the extra delay time for the mapping signals as needed. The parsed mapping table contains delay as well. The format is: <full path of target signal> #<delay value> The delay value following the “#” operator can be a positive or a negative integer. Following is an example for the delay file: <pre>top.clk #-10 top.rst #10</pre> By default, the time unit in the delay file is identical to the time unit of the input FSDB. If you want to use different time unit in the delay file, they can specify it in the first line of the delay file as follows: <pre>// time_unit <s ms us ns ps fs></pre> Following is an example of the delay file with the time unit: <pre>// time_unit ns top.clk #-0.01 top.rst #0.01</pre> NOTE: Even if a delay file has its own time unit, the delay value is finally converted to an integer according to the time unit of the input FSDB. The configuration is only used for replay simulation. Example: <pre>set delay = ./delay.map</pre>

SDF	Optional. The SDF file is annotated when running the simulation if this data is specified. The format is as follows: <pre>set SDF = "<SDF_file_path>" "<corresponding_instance>" "" "<SDF_log_file>" "<mtm_spec>"</pre> Even if the SDF file is included in the original testbench, it is not extracted by the <code>wisim</code> utility automatically. You have to specify the SDF file in the configuration file manually. The default MTM for the SDF file is typical. It needs to be specified if the MTM of SDF file is not typical. Multiple SDF files can be specified in the configuration file. Example: <pre>set SDF = "../Design/CPU_max.sdf" "system.i_cpu" "" "SDF_log1.txt" "MAXIMUM" "" set SDF = "../CPU_max2.sdf" "system.i_cpu_02" "" "" "MINIMUM"</pre>
Design	Optional. The configuration loads the target design with the Tcl command file. Alternatively, specify design importing options on the <code>wisim</code> command line. For example, <pre>%> wisim -sv -f run.f -config config_file -replay_extract</pre> Example: <pre>set Design = ./system_load.command</pre>
Exclude_Module_File	Optional. The configuration specifies a list of module names to exclude into a file. During the extraction phase, those modules are empty modules with module ports and signals only. If the specified modules are not found, a warning message is displayed. If a top module is specified in the file list of excluded modules, the top module is not excluded. The following is an example for the file of excluded modules: <pre>MODULE1 MODULE2 MODULE3</pre> NOTE: The signals of the excluded modules should not be mapped in the mapping file. Example: <pre>set Exclude_Module_File = ./exc_mod.txt</pre>
FSDB_HIER_MAP	Optional. Use this configuration option, if the mapping file's RHS hierarchy is different from RTL FSDB. Example: <pre>set FSDB_HIER_MAP = dut.usub.my_register tb.udut.usub.my_register</pre>

In the current mapping flow, you must set the scope (in the “set Scope” configuration) carefully according to the mapping file. The `wisim` utility extracts the design only under the specified scope so the signals that are not included in the scope are ignored.

Note:

All configuration keywords in the configuration file are not case sensitive.

Text Mapping File

The format of the text mapping file (used by the “set Map” configuration) is shown below:

```
<target signal with full hierarchical name> => <source signal with full hierarchical name>
```

Or

```
<target signal with full hierarchical name> => <constant>
```

The “=>” operator can be replaced by the “~>” operator for inverse-mapping. The mapped value is inversed if the “~>” operator is used in the mapping file. The following is an example of the mapping file:

```
target_top.u1.u2.u3.t_sig1 => source_top.u1.u2.u3.s_sig1
target_top.u1.u2.u3.t_sig2 ~> source_top.u1.u2.u3.s_sig2
target_top.u1.u2.u3.g_sig3 => 1'b1
```

The following rules must be considered when creating the mapping file:

- The prefix (“1'b” in above example) must be added for a constant value.
- The scope delimiter is fixed to ‘.’.
- By default, the `wisim` utility forms the bus-to-bus mapping. See the Form Bus Flow section for the form bus flow details.
- All target signals must be inside the target scope.

Form Bus Flow

If a bit selection of a bus exists in the mapping file, the `wisim` utility forms the bus-to-bus mapping if it is possible. All bits of a gate-level bus must be mapped to the same RTL bus if any bit is mapped to a bit of an RTL bus. Equivalently, all bits of a target bus must be mapped to a constant if any bit is mapped to a constant. Assuming T is a 2-bit bus, the bus can be mapped as follows:

```
T[0] => C[0]
T[1] => C[1]
```

When a bus-to-bus mapping fails, the following is the flow for forming a bus in replay simulation:

1. If it fails to form a bus, the `wisim` utility keeps the bit-level mapping. The following are possible reasons that cause the `wisim` utility to fail to form the bus:

- a. The mapping for a bus is not completely described:

```
T[0] => B[0] // T[1] is not specified in the mapping file
```

- b. The mapping for a bus cannot be a bus in the waveform part:

```
T[0] => B[0]
T[1] => C[0]
```

- c. The mapping for a bus contains both inverted and non-inverted mappings:

```
T[0] => B[0]
T[1] ~> B[1]
```

- d. The mapping for a bus contains both constant and non-constant mappings:

```
T[0] => B[0]
T[1] => 1'b1
```

2. To drop the bus signals that cannot be formed as the bus-to-bus mapping, set the `WISIM_DROP_BUS_BIT` environment variable to 1:

```
setenv WISIM_DROP_BUS_BIT = 1
```

Then, the signals that meet failure cases in the flow are dropped and not forced.

3. To skip the flow to form a bus, set the `WISIM_SKIP_FORM_BUS` environment variable to 1:

```
setenv WISIM_SKIP_FORM_BUS = 1
```

Then, replay simulation does not form a bus. But, it still keeps the valid mapping description in the mapping files.

Simulation Compile Script Examples

You need to create and specify the Simulation Compile script by the “set Simulation Compile Script” configuration in the configuration file. The following is an example of the Simulation Compile script for VCS:

```
#!/bin/csh -f
setenv VCS_HOME <VCS_INSTALL_PATH>
setenv PATH $VCS_HOME/bin:$PATH
setenv PLATFORM LINUX64
setenv LD_LIBRARY_PATH
    ${VERDI_HOME}/share/PLI/VCS/${PLATFORM}:$LD_LIBRARY_PATH
setenv TAB ${VERDI_HOME}/share/PLI/VCS/${PLATFORM}/novas.tab
```

```
setenv PLI ${VERDI_HOME}/share/PLI/VCS/${PLATFORM}/pli.a

### COMMAND ###
vcs -full64 -sverilog -f wi_run.f -P $TAB $PLI +cli+4 -debug_pp

### debug_access flow ###
#setenv NOVAS_HOME $VERDI_HOME
#vcs -sverilog -f wi_run.f -debug_access+f+w+cbk+fwn
  -debug_region=cell+lib
```

The `wi_run.f` file in the replay simulation directory is generated by the `wisim` utility for the required design scope and new testbenches.

The following is an example of the Simulation Compile script for IUS:

```
#!/bin/csh -f

## ncsim flow ##
setenv CDS_INST_DIR <IUS_INSTALL_PATH>
setenv PATH ${CDS_INST_DIR}/tools/bin:$PATH
setenv PLATFORM LINUX64
setenv LD_LIBRARY_PATH
  ${VERDI_HOME}/share/PLI/IUS/${PLATFORM}/boot:${VERDI_HOME}/share/PLI/I
  US/${PLATFORM}:$LD_LIBRARY_PATH
#ncvlog -64BIT -SV -f wi_run.f
#ncelab -64BIT -NOTIMINGCHECKS -SDF_NO_WARNINGS -NOWARN CUVWSP WI_TB_TOP
  designTop -access +rw

## ncverilog flow ##
#ncverilog +nc64bit +notimingcheck +nowarn+CUVWSP +nowarn+SDFINC -sv -f
  wi_run.f +access+rw -c

## irun flow ##
irun -64bit +notimingcheck -SDF_NO_WARNINGS -sv -f wi_run.f -access +rw
  -c
```

The `wi_run.f` file is generated by the `wisim` utility (see the details in VCS script example). The top module `WI_TB_TOP` (highlighted in red) is a fixed name and has to be placed before the original design top (`designTop` in this example).

Simulation Run Script Examples

You need to create and specify the Simulation Run script by the `set Simulation_Run_Script` configuration in the configuration file. The following is an example of the Simulation Run Script for VCS:

```
#!/bin/csh -f
setenv PLATFORM LINUX64
setenv LD_LIBRARY_PATH
  ${VERDI_HOME}/share/PLI/VCS/${PLATFORM}:$LD_LIBRARY_PATH

./simv +notimingcheck
```

The following is an example of the Simulation Run script for IUS:

```
#!/bin/csh -f
setenv CDS_INST_DIR <IUS_INSTALL_PATH>
setenv PATH ${CDS_INST_DIR}/tools/bin:$PATH
setenv PLATFORM LINUX64
setenv LD_LIBRARY_PATH
  ${VERDI_HOME}/share/PLI/IUS/${PLATFORM}/boot:${VERDI_HOME}/share/PLI/I
  US/${PLATFORM}:$LD_LIBRARY_PATH

## ncsim flow ##
#ncsim -64BIT WI_TB_TOP

## ncverilog flow ##
#ncverilog +nc64bit -R

## irun flow ##
#irun -64bit -R
```

The top module `WI_TB_TOP` is a fixed name for ncsim.

Generate Template Files

You can use the following command to generate template files for wisim flow:

```
%>wisim -gen_template [VCS|IUS]
```

For example, using the command `wisim -gen_template VCS` for VCS generates the following three files under the `WISIM_TEMPLATE/` directory:

1. `wi_config`: the template configuration file for wisim
2. `vcs_wi_compile.rc`: the template VCS compilation script for wisim
3. `vcs_wi_run.rc`: the template VCS run script for wisim

Supported Platforms and Simulator

The supported platforms are LINUX32/64 and SUSE32/64. The supported simulators are Synopsys VCS and Cadence IUS.

Practice

This practice section demonstrates how to use the `crdb` and `wisim` utilities to conduct a gate-level (GTL) simulation from the RTL FSDB file.

- Input:

RTL design (for `crdb`)

RTL FSDB (for `wisim`)

GTL design (for both `crdb` and `wisim`)

- Output:

GTL simulation result (performing `crdb` for G2R mapping and `wisim` for replay simulation)

Step1: Generate Mapping File

1. Change the directory to `DEMO/CORR/`.

2. Prepare two designs:

Compile RTL design to `rtl.lib++`: `vericom -f run_rtl.f -lib rtl`

Compile GTL design to `gate.lib++`: `vericom -f run_gate.f -lib gate`

3. Generate the mapping file with the `crdb` utility by using the following commands:

```
crdb -RTL "-lib rtl -top system" -GATE "-lib gate -top system" -crdb
g2r.crdb -expMap g2r.map.gz -allreg -top_inport -target_scope system
-expGateUncor -impSVF design.svf
```

The following are descriptions of these commands:

- `-expMap`: Output the mapping file in the `gzip` format.
- `-allreg`: Map all registers.
- `-top_inport`: Map all input and inout signals of the target scope.
- `-target_scope`: Specify the target scope.
- `-impSVF`: Specify the `.svf` file.
- `-expGateUncor`: Export uncorrelated signals.

4. Check the following files to review the correlation results:

- `crdbLog/crdb.log` file shows the correlation ratio similar to:

```
Correlation Statistics - totally 110 item(s)
-----
signal(s)  105    correlated 105    (100%)
  bit(s) 105    correlated 105    (100%)
split signal(s) 5
  bit(s) 19    correlated 19    (100%) (not in crdb 0, illegal 0)
total bit(s) 124    correlated 124    (100%)
instance(s)  0    correlated 0
not in crdb  0    invalid  0
```

- `g2r.map.gz` file is the mapping results in the `gzip` format.

Step2: Replay Simulation

1. Change the directory to `DEMO/VCS/`
2. Check the configuration file, `wi_config`.

```
set FSDB      =    ./rtl.fsdb
set Scope     =    system
set Map       =    ./g2r.map.gz
set Begin_Time =    1000
set End_Time  =    5000
set Time_Unit =    ns

set Simulation_Compile_Script =    ./vcs_wi_compile.rc
set Simulation_Run_Script    =    ./vcs_wi_run.rc

set SDF       =    "../Gate_Design/CPU.sdf" "system.i_cpu" "" "" "TYPICAL"
```

3. Modify `VCS_HOME` in the `vcs_wi_compile.rc` file to the VCS install path.
4. Set the correct `VERDI_HOME` to VERDI install path.
5. Perform the replay extraction using the following command:

```
%>wisim -lib gate -config wi_config -replay_extract
```

- All required data for wisim is generated under `whatlf/`.

6. Perform replay compilation:

```
%>wisim -config wi_config -replay_compile
```

- An executable for simulation is generated as `whatlf/simv`.

7. Perform replay simulation:

```
%>wisim -config wi_config -replay_sim
```

- The simulation result (`wi_result.fsdb`) is generated under `whatIf/`.

Known Problems and Limitations

The following are the known problems and limitations:

- The signals in the mapping file cannot be a part/bit selection of a packed structure.
- The following data types are not supported in the left-hand side of the mapping file for IUS simulation:
 - **Integer types:**
 1. `shortint`
 2. `int`
 3. `longint`
 4. `byte`
 5. `integer`
 6. `time`
 - **enum**
 - **real**
 - **SVTB signals**
- For IUS simulator, FSDB dumping may be incorrect in IUS10.2 and IUS11.1 versions. It is recommended to use other IUS simulator versions, such as IUS12.1 for replay simulation FSDB dumping. Alternatively, the `FSDB_IUS_NO_OPTIM` environment variable is provided for IUS10.2 and IUS11.1 versions to avoid this issue:

```
setenv FSDB_IUS_NO_OPTIM 1
```

- For the protected code, replay simulation analysis does not support the target scope and its sub-scope that includes the protected code.